Sample-based planning

杜少鹤

Planning problem definition (without constraint)

1. A *world* $\mathcal{W}$ in which either $\mathcal{W} = \mathbb{R}^2$ or $\mathcal{W} = \mathbb{R}^3$.

2. A semi-algebraic *obstacle region* $\mathcal{O} \subset \mathcal{W}$ in the world.

3. A semi-algebraic *robot* is defined in $\mathcal{W}$. It may be a rigid robot $\mathcal{A}$ or a collection of m links, $\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_m$.

4. The *configuration space* $\mathcal{C}$ determined by specifying the set of all possible transformations that may be applied to the robot. From this, $\mathcal{C}_{obs}$ and $\mathcal{C}_{free}$ are derived.

5. A configuration, $q_I \in \mathcal{C}_{free}$ designated as the *initial configuration*.

6. A configuration $q_G \in \mathcal{C}_{free}$ designated as the *goal configuration*. The initial and goal configurations together are often called a *query pair* (or *query*) and designated as (q_I, q_G).

7. A complete algorithm must compute a (continuous) *path*, $\tau : [0, 1] \rightarrow \mathcal{C}_{free}$, such that $\tau(0) = q_I$ and $\tau(1) = q_G$, or correctly report that such a path does not exist.

Explicitly represent C_{obs} is expensive

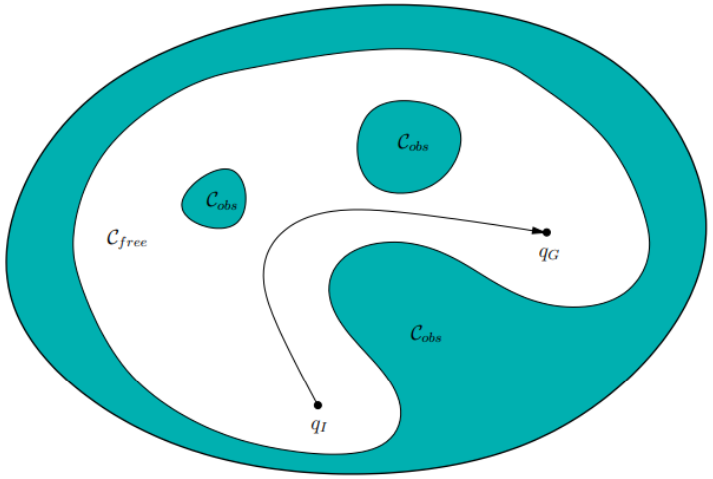
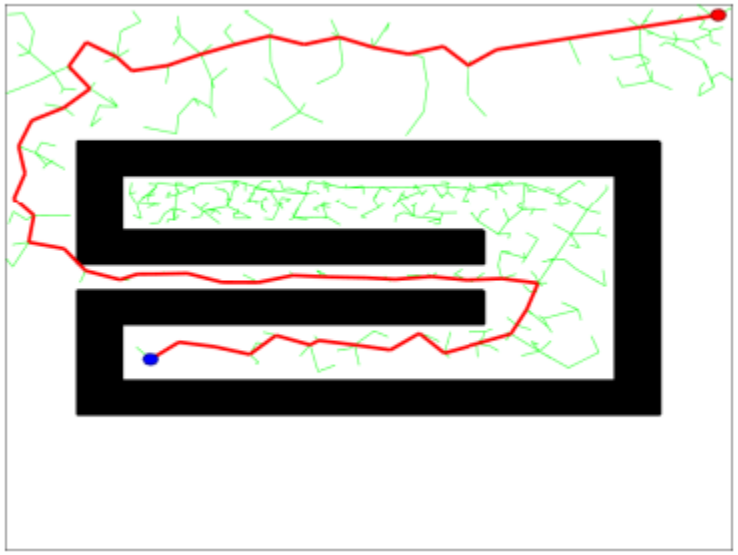


Figure 4.11: The basic motion planning problem is conceptually very simple using C-space ideas. The task is to find a path from q_I to q_G in $\mathcal{C}_{free}$. The entire blob represents $\mathcal{C} = \mathcal{C}_{free} \cup \mathcal{C}_{obs}$.

Challenges

1. high-dimensional 计算复杂度 空间复杂度指数上升 2^{in}
2. Narrow-corridors
3. Local-minimum
4. Explicitly represent C-obs is expensive



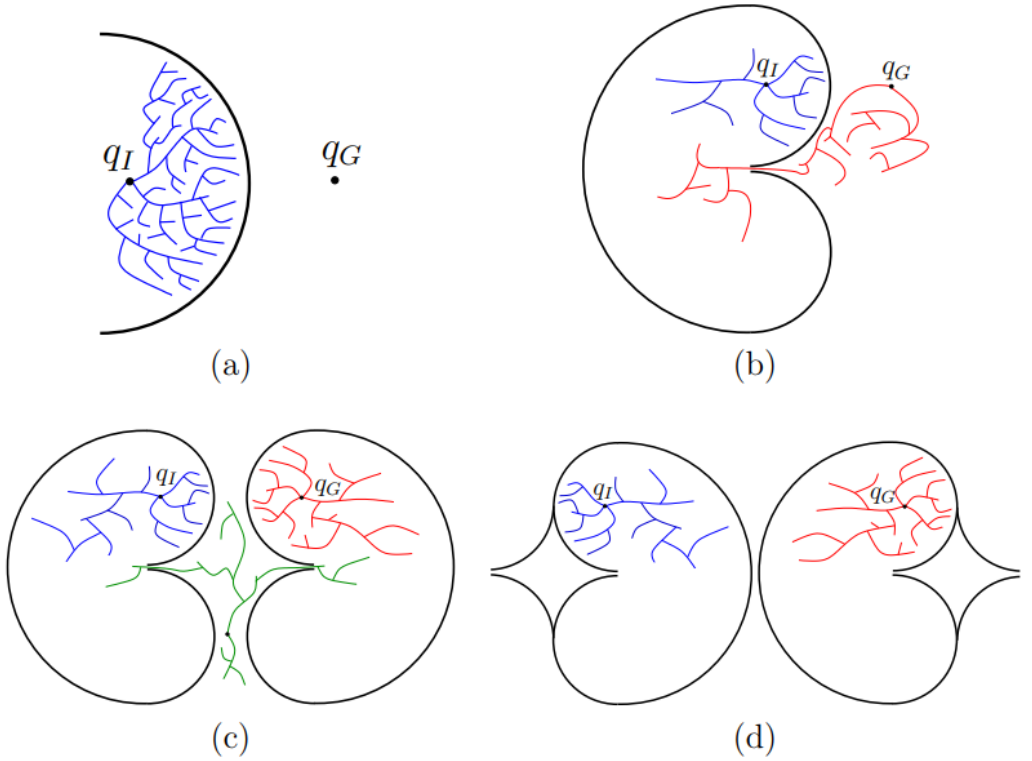


Figure 5.13: All of these depict high-dimensional obstacle regions in C-space. (a) The search must involve some sort of multi-resolution aspect, otherwise, that algorithm may explore too many points within a cavity. (b) Sometimes the problem is like a bug trap, in which case bidirectional search can help. (c) For a double bug trap, multi-directional search may be needed. (d) This example is hard to solve even for multi-directional search.

Sample-Based Motion Planning

Single-query
Multiple-query

Graph-based
Tree-based
Has loop?

Unchanging-environment
Changing-environment

Precomputation
online

Probabilistic Roadmap Method (PRM)

Rapidly-Exploring Random Tree (RRT)、RDT
Potential Fields

4

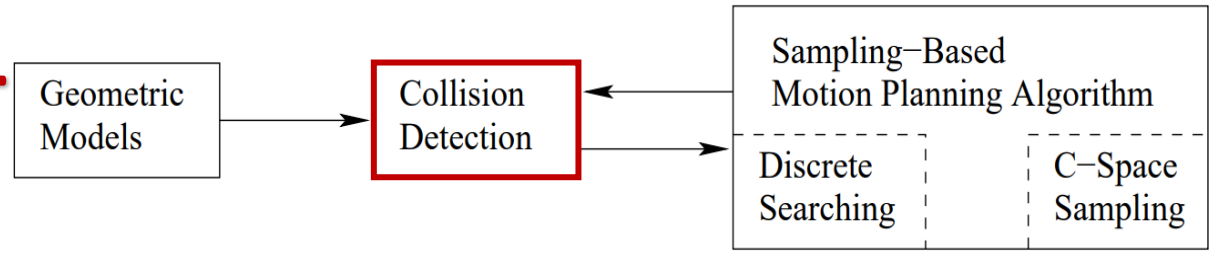


Figure 5.1: The sampling-based planning philosophy uses collision detection as a "black box" that separates the motion planning from the particular geometric and kinematic models. C-space sampling and discrete planning (i.e., searching) are performed.

There are two general ways to avoid having to select this resolution (or more generally, dispersion):

1. Iteratively refine the resolution until a solution is found. In this case, sampling and searching become interleaved. One important variable is how frequently to alternate between the two processes. This will be presented shortly.

1

2. An alternative is to abandon the adaptation of discrete search algorithms and develop algorithms directly for the continuous problem. This forms the basis of the methods in Sections 5.4.3, 5.4.4, and 5.5.

dispersion、discrepancy

3

Random Walk, muti-tree

Sample-Based Motion Planning

Algorithms template(Single-query)

1. **Initialization:** Let $\mathcal{G}(V, E)$ represent an undirected *search graph*, for which V contains at least one vertex and E contains no edges. Typically, V contains q_I, q_G, or both. In general, other points in $\mathcal{C}_{free}$ may be included.

2. **Vertex Selection Method (VSM):** Choose a vertex $q_{cur} \in V$ for expansion.

3. **Local Planning Method (LPM):** For some $q_{new} \in \mathcal{C}_{free}$ that may or may not be represented by a vertex in V, attempt to construct a path $\tau_s : [0, 1] \rightarrow \mathcal{C}_{free}$ such that $\tau(0) = q_{cur}$ and $\tau(1) = q_{new}$. Using the methods of Section 5.3.4, τ_s must be checked to ensure that it does not cause a collision. If this step fails to produce a collision-free path segment, then go to step 2.

4. **Insert an Edge in the Graph:** Insert τ_s into E, as an edge from q_{cur} to q_{new}. If q_{new} is not already in V, then it is inserted.

5. **Check for a Solution:** Determine whether $\mathcal{G}$ encodes a solution path. As in the discrete case, if there is a single search tree, then this is trivial; otherwise, it can become complicated and expensive.

6. **Return to Step 2:** Iterate unless a solution has been found or some termination condition is satisfied, in which case the algorithm reports failure.

VSM LPM 衍生出不同的规划算法
带Constraint 问题也是主要处理VSM LPM，使得VSM 选择的vertex，LPM 生成的edge在constraint function的manifold上

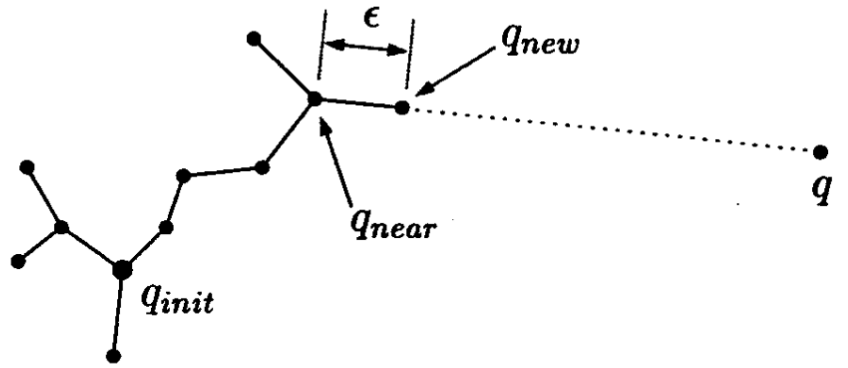
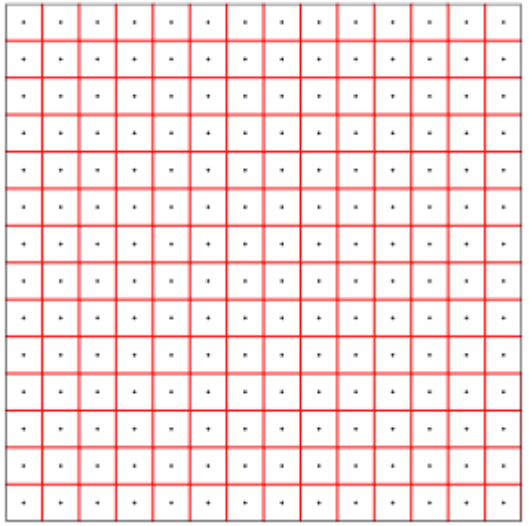


Figure 3: The EXTEND operation.

Grid-Resolution + discrete search

approximating C-space with a high-resolution grid

- high-dimensional 计算复杂度 空间复杂度指数上升
- Local-minimum



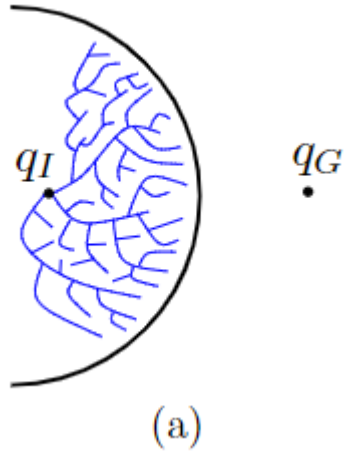
trade-off resolution VS efficiency

Grid resolution issues The method explained so far will nicely find the solution to many problems when provided with the correct resolution. If the number of points per axis is too high, then the search may be too slow. This motivates selecting fewer points per axis, but then solutions might be missed. This trade-off is fundamental to sampling-based motion planning. In a more general setting, if other forms of sampling and neighborhoods are used, then enough samples have to be generated to yield a sufficiently small dispersion.

There are two general ways to avoid having to select this resolution (or more generally, dispersion):

1. Iteratively refine the resolution until a solution is found. In this case, sampling and searching become interleaved. One important variable is how frequently to alternate between the two processes. This will be presented shortly.

2. An alternative is to abandon the adaptation of discrete search algorithms and develop algorithms directly for the continuous problem. This forms the basis of the methods in Sections 5.4.3, 5.4.4, and 5.5.



Randomized Potential Fields

Best First VSM: most recently v added to the *tree*
Best First LPM: create a new vertex Vn, in a neighborhood of v, in a direction that minimizes potential function
Random Walk VSM: most recently v added to the *tree*
Random Walk LPM: generates a direction and magnitude randomly.
Backtrack: selects a vertex at random from among the vertices in V that were obtained during a random walk
Stuck: after some number of tries, if LPM is unsuccessful at reducing potential function

将C-space高精度栅格化，
∇q respect resolution

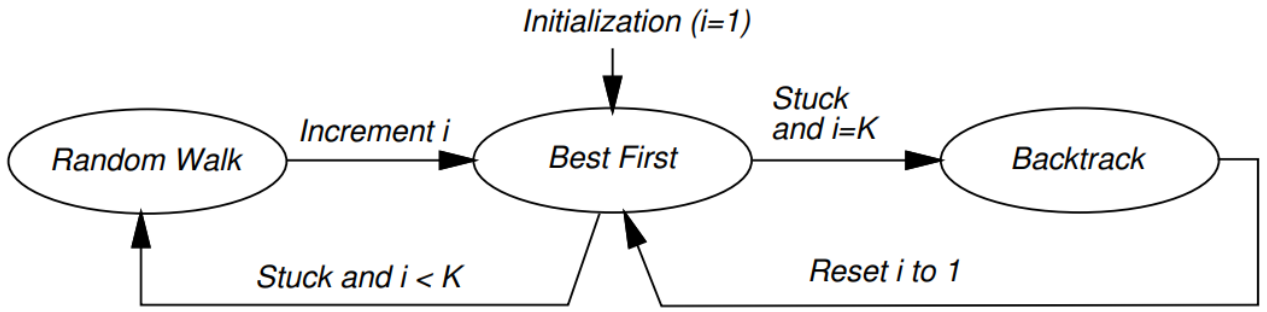


Figure 5.15: The randomized potential field method can be modeled as a three-state machine.

problem: paths are often too complicated to be useful in applications(simplification)
drawbacks: parameter tuning; narrow corridors

RRT

TREEPLANNER(q_{start}, q_{goal})
  $\mathcal{T}$.init(q_{start});
  while no path from q_{start} to q_{goal} **do**
    $q_{\text{rand}} \leftarrow$ Sample();
    $q_{\text{near}} \leftarrow$ Select($\mathcal{T}$, q_{rand})
    $q_{\text{new}} \leftarrow$ Extend(q_{near}, q_{rand});
    if Connect(q_{new}, q_{near}) **then**
      $\mathcal{T}$.Add(q_{near}, q_{new})

VSM: q_{rand} 最近的点q_{near}
LPM: 朝着 q_{near} --> q_{rand} 方向 *extend a stepsize*

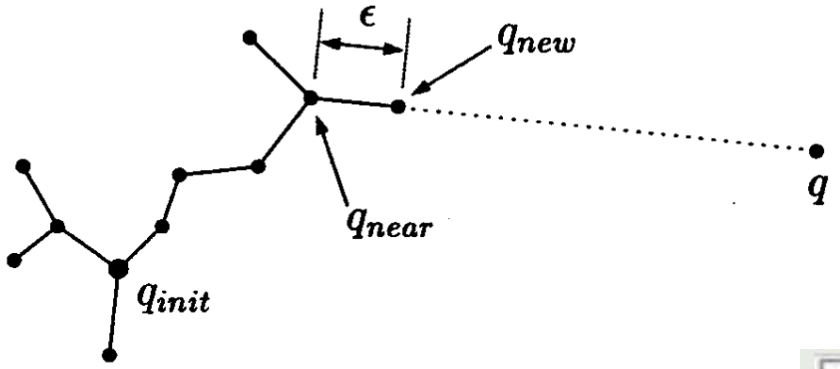
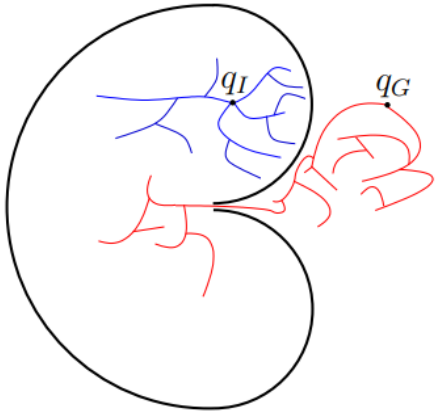
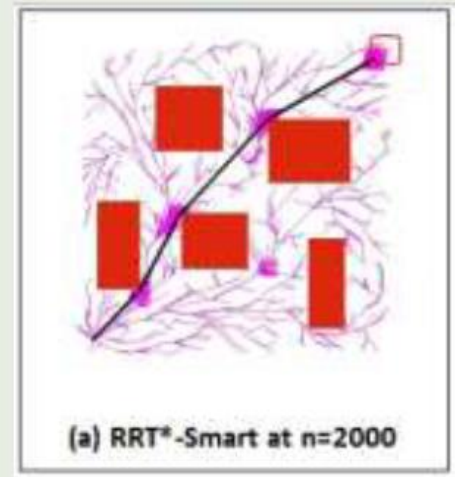


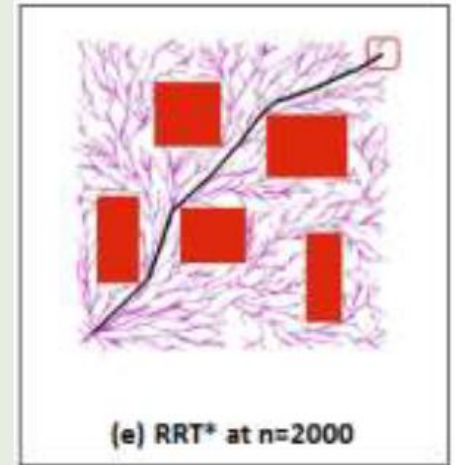
Figure 3: The EXTEND operation.



(b)



(a) RRT*-Smart at n=2000



(e) RRT* at n=2000

PRM

Precomputation, Unchanging-environment

```
GraphPlanner(q_start, q_goal)
    G.init(q_start, q_goal);
    while no path from q_start to q_goal do
        q_rand ← Sample();
        Q ← SelectNghbrs(G, q_rand)
        for all q_near ∈ Q do
            if Connect(q_near, q_rand) then
                G.Add(q_near, q_rand)
```

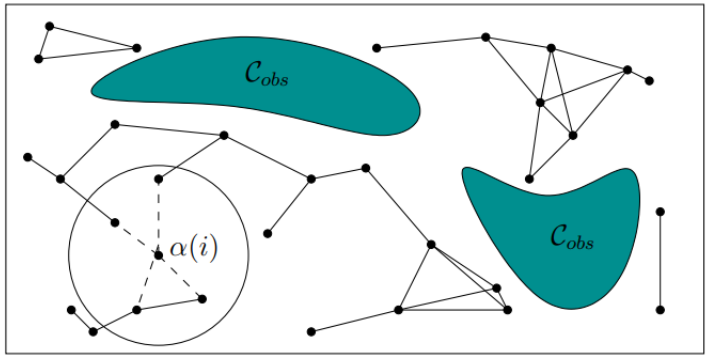


Figure 5.26: The sampling-based roadmap is constructed incrementally by attempting to connect each new sample, $\alpha(i)$, to nearby vertices in the roadmap.

Planning problem definition (with constraint)

Constraint function: F(q) = 0
Assumption: J(q) is full rank; F(q) is C^2-smooth (smooth manifold() without singularity)

Motion Planning problem with constraint:
Find a continuous path
t:[0,1] → M ∩ C-free
t(0) = qi, t(1) ∈ $\mathcal{Q}g$

VSM: sample vertex in manifold;
LPM: edge between two points is in manifold

Metric：nearest-neighbor 理论上应用 geodesic 距离，实际用C-space metric近似 geodesic（二分法判断是否适用）

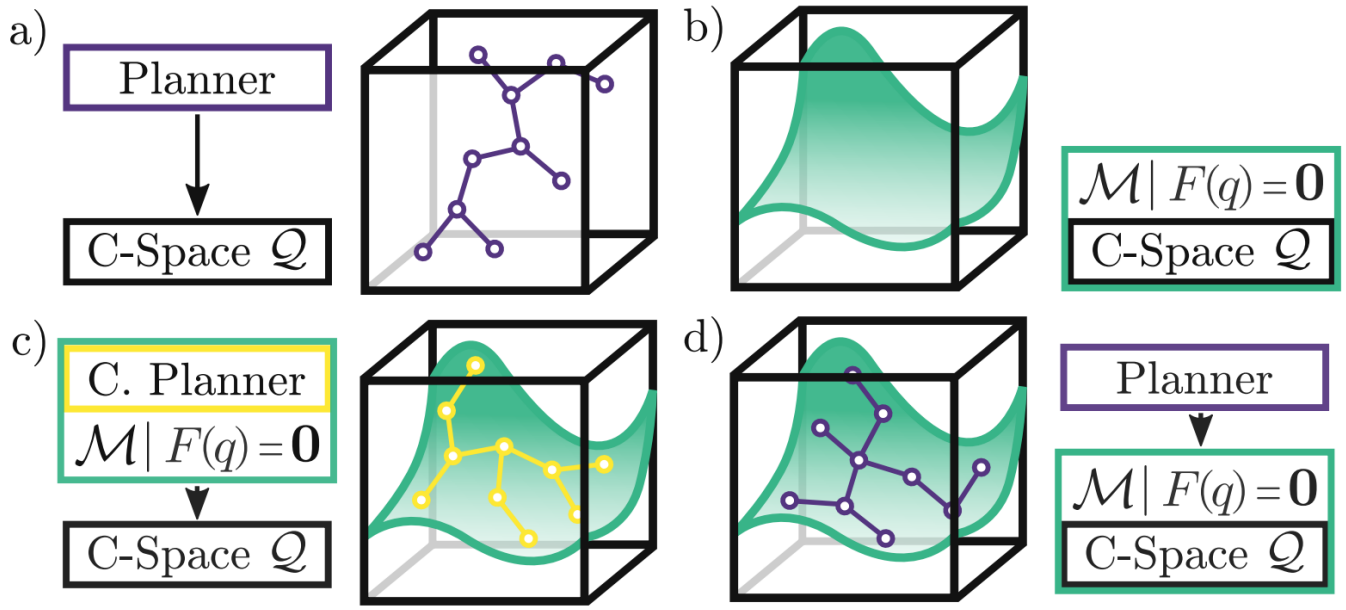


Figure 2. A depiction of the framework and its relation to sampling-based planners. **a)** A box configuration space $\mathcal{Q}$ is shown in black. A sampling-based planner (purple) plans in $\mathcal{Q}$ using primitives afforded by the space. **b)** A constraint function $F(q) = \mathbf{0}$ defines a implicit manifold $\mathcal{M}$ (green). **c)** An augmented constrained sampling-based planner (yellow) (e.g., CBIRRT2, etc.) plans on $\mathcal{M}$, using its constraint methodology. **d)** IMACS enables *any* sampling-based planner (purple) to plan on $\mathcal{M}$ by incorporating $\mathcal{Q}$ and the constraint function $F(q) = \mathbf{0}$.

Methodologies(constraint)

- Relaxation → Moveit
- Projection → OMPL
- Tangent Space → OMPL
- Atlas → OMPL
- Reparameterization (mapback)

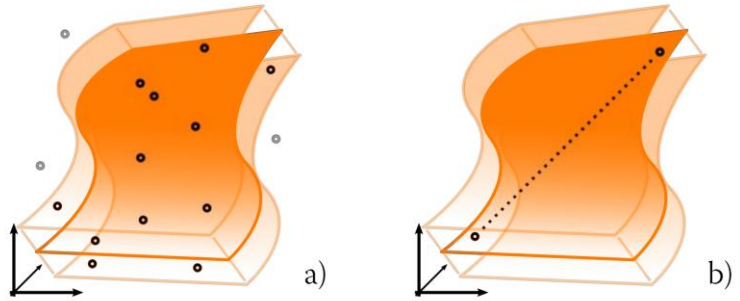


Figure 5

Sampling and local planning with relaxation-based constraint handling. The constraint manifold (orange) is given non-zero volume by relaxing the constraint according to some tolerance (boundaries are shown by faded extensions of the manifold). **a)** Standard rejection sampling is done to find close-to-satisfying configurations. Invalid samples are in grey, valid samples in black. **b)** Standard local planning is done (dashed line).

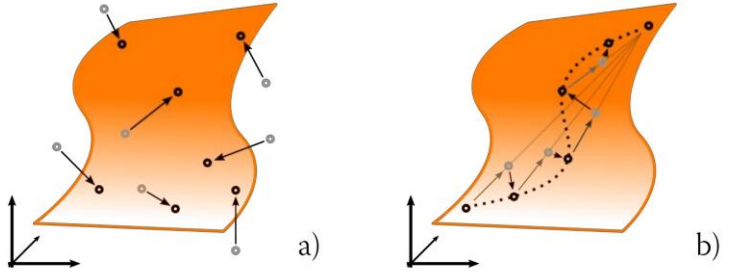


Figure 6

Sampling and local planning with projection-based constraint handling. The constraint manifold (orange) is projected to (black arrows) using a projection operator. **a)** After drawing a sample from configuration space (grey), it is retracted to the surface of the constraint manifold (black) using the projection operator. Local planning is shown in **b)**. From a starting configuration (bottom left), a new unsatisfying configuration (grey) closer to the goal is generated by interpolation (grey arrow). That configuration is then projected to the manifold (black arrow), and the process continues till the goal is reached or another termination condition is met.

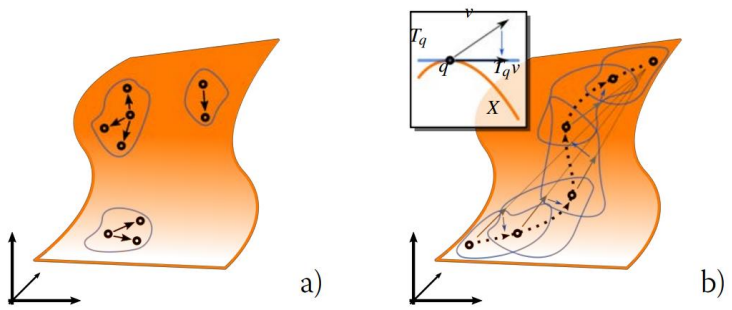


Figure 7

Tangent space-based constraint handling. Given an initial satisfying configuration, a tangent space to that point can be computed which is the nullspace of the constraint function's Jacobian (blue blobs), and new satisfying configurations can be generated by local perturbations (black arrows). **a)** samples are generated by creating a tangent space at a known configuration and projecting random vectors. Local planning is shown in **b)**. From a starting configuration (bottom left), a tangent space is computed, and the vector from the current configuration to the goal is computed v (grey arrow) and projected (blue arrow) into the tangent space (black arrowhead). The projected vector is added to the current configuration to generate a novel configuration close to satisfying the constraint.

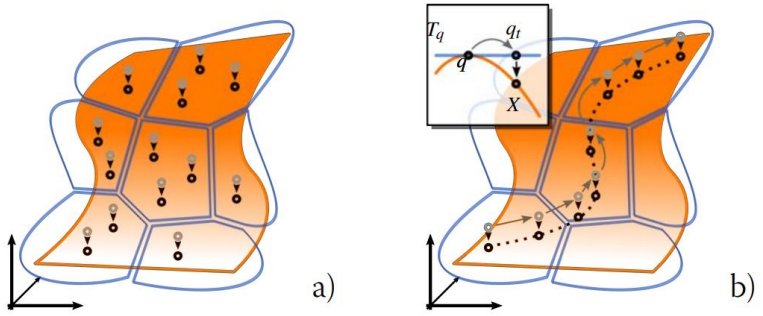


Figure 8

Atlas-based sampling and local planning, akin to AtlasRRT. The atlas is a set of tangent polytopes covers the constraint manifold (blue blobs). In this figure, the atlas has already been computed and covers the space. During planning, the atlas is constructed in tandem with sampling and local planning. **a)** sampling of the manifold is done by drawing samples from the tangent polytopes by randomly sampling within (grey). These points are then orthogonally projected from polytopes to the surface of the manifold (black arrow to black), ψ_q. Local planning is shown in **b)**. Roughly, interpolation is done with the tangent space (grey arrows) from a configuration q to another q_t, and new configurations are projected to the manifold for validation (black arrows). This continues until the goal is reached. See (31) for details.

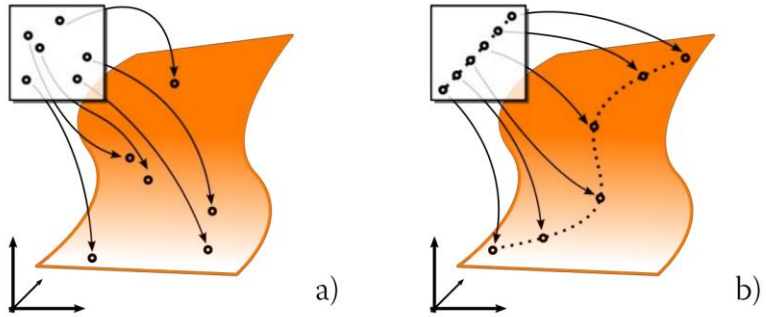


Figure 10

Reparameterization-based constrained handling. A new configuration space is computed from the manipulator and constraint, reparameterizing the space (white square). **a)** samples can be draw from the reparameterized space and mapped back into the original configuration space (black arrows). Local planning is shown in **b)**. Interpolation is done within the reparameterized space and mapped back into the original configuration space.

Projection Operator

带约束的优化问题

$$\begin{aligned} \min_{q} \quad & \frac{1}{2}\|q_0 - q\|_2{}^2 \\ \text{subject to} \quad & F(q) = \mathbf{0} \end{aligned}$$

引入拉格朗日算子变为无约束的优化问题
可用梯度下降法求解

$$\Lambda(q, \lambda) = \frac{1}{2}\|q_0 - q\|^2 - F(q)^T \lambda$$

$q \in C, P(q) = q_m, \quad q_m \in M;$

Algorithm 2 P, a Projection Operator

Input q, an initial configuration to project from
Output $P(q) \in \mathcal{M}$, the projected state configuration. On failure, "false" is returned.

```
1: procedure P(q)
2:     x ← F(q)
3:     while ‖x‖_2 > ε and iterations remain do
4:         q ← q − J(q)^+ x
5:         x ← F(q)
6:     if ‖x‖_2 ≤ ε then
7:         return q
8:     else
9:         return false
```

Projection

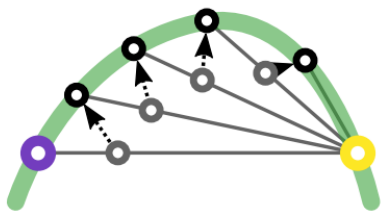
assume a sampler s is non-zero density distribution over C.

- Projection-based sampler(VSM): repeatedly sample a point using the sampler from the ambient configuration space and project the point.
- Local Planner(LPM)： Projection Integration

IK(projection operator): 约束在SE(3)一点，

constraint function: Fk(q) = P, self-motion manifold

a)



Algorithm 3 Projection Integrator

Input $q_a \in \mathcal{M}$, an initial configuration, and $q_b \in \mathcal{Q}$, a goal configuration.

Output $\{q_k\}_{k=1}^{j}$ such that $\forall k, q \in \mathcal{M}$. Upon success, $\|q_b - q_j\|_2 \leq s$.

procedure INTEGRATE(q_a, q_b)
  $q_1 \leftarrow q_a, j \leftarrow 1$
  while $\|q_j - q_b\|_2 > s$ **do**
    // Interpolate in $\mathcal{C}$-space
    $q_{j+1} \leftarrow P(q_j + (s/\|q_b - q_j\|_2)(q_b - q_j))$
    $d \leftarrow \|q_{j+1} - q_j\|_2$
    // Check distortion & progress
    if $d > \lambda s$ or $d < s$ **then**
      break
    $j \leftarrow j + 1$
  return $\{q_k\}_{k=1}^{j}$

Atlas

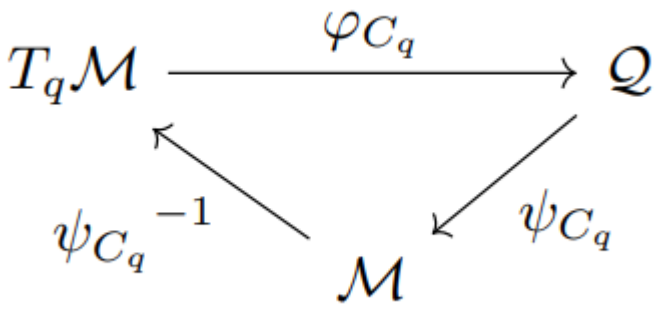
- Tangent space： null space of the Jacobian of constraint function

$$F(q) = 0, J(q)\dot{q} = 0$$

- chart:

$$C_q = (q, \Phi(q)) \qquad \begin{bmatrix} J(q) \\ \Phi(q)^T \end{bmatrix} \Phi(q) = \begin{bmatrix} \mathbf{0} \\ I \end{bmatrix}$$

- Chart ←→ manifold:



$$q_u = \varphi_{C_q}(u_q) = q + \Phi(q)u_q$$

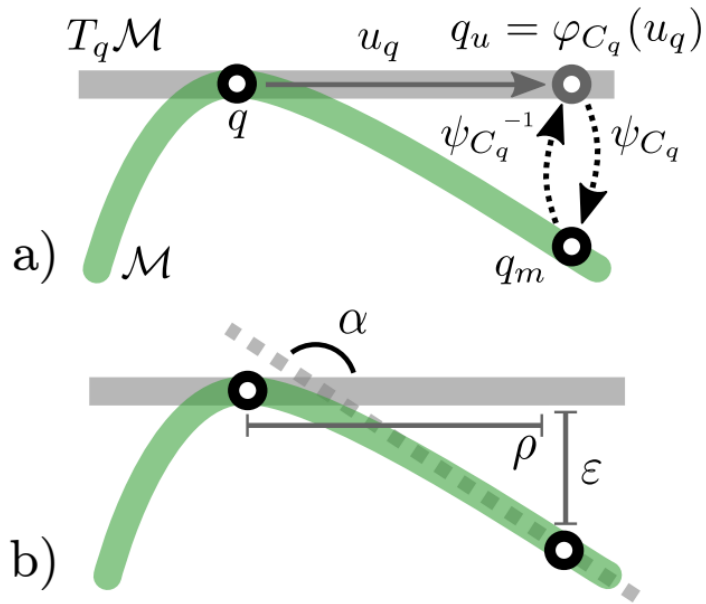
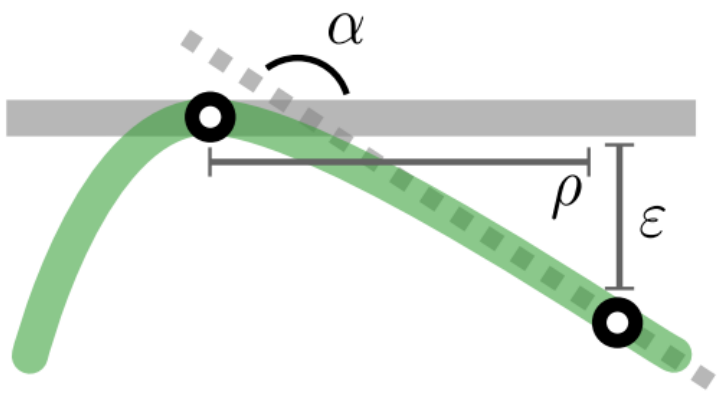


Figure 4. a) Illustration of the various chart operators as described in Section 4.3.1. **b)** Illustration of the parameters used in a chart's validity region.



- ε: maximum distance of $T_q\mathcal{M}$ to $\mathcal{M}$
- ρ: radius of ball in $T_q\mathcal{M}$
- α: maximum curvature of $T_q\mathcal{M}$ to $\mathcal{M}$

The following inequalities define the validity region, for a point $u_q \in T_q\mathcal{M}$, $q_u = \varphi_{C_q}(u_q)$, $q_m = \psi_{C_q}(q_u)$:

$$\|q_u - q_m\|_2 \leq \varepsilon \quad \frac{\|u_q\|_2}{\|q - q_m\|_2} < \cos(\alpha) \quad \|u_q\|_2 \leq \rho$$

Algorithm 4 ψ_{C_q}, the exponential map

Input q_u, an initial configuration from C_q's tangent space

Output $q_m = \psi_{C_q}(q_u)$, the orthogonally projected state configuration. On failure, "false" is returned.

// $A_u(q_m), b_u(q_m)$ defined in Eq. 4

procedure $\psi_q(q_u)$
  $q_m \leftarrow q_u$
  $b \leftarrow b_u(q_m)$
  // Orthogonally project until convergence
  while $\|b\|_2 > \epsilon$ **and** iterations remain **do**
    $q_m \leftarrow q_m - A_u(q_m)^{-1} b$
    $b \leftarrow b_u(q_m)$
  if $\|b\|_2 \leq \epsilon$ **then**
    return q_m
  else
    return false

Projection operator

$$A_u(q_m) = \begin{bmatrix} J(q_m) \\ \Phi(q)^T \end{bmatrix}, \quad b_u(q_m) = \begin{bmatrix} F(q_m) \\ \Phi(q)^T (q_m - q_u) \end{bmatrix}$$

bu: constraint function; Au: the Jacobian of bu

Create Charts

IMACS implementation: atlas is stored as a collection of chart within a nearest neighbor structure, using the configuration space's metric between chart centers.

Algorithm 5 Find and Create Chart for q_a in $\mathcal{A}_\mathcal{M}$

Input q_a, the configuration to find a chart for, and $\mathcal{A}_\mathcal{M}$, the current atlas of charts

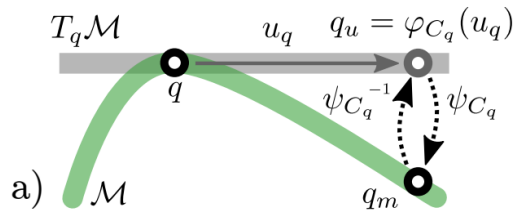
Output C, the best (distance-wise) chart for q_a. On failure (no valid charts for q_a), an empty object $\{\}$ is returned.

1: **procedure** FINDCHART(q_a, $\mathcal{A}_\mathcal{M}$)
2: best $\leftarrow \varepsilon$, $C \leftarrow \{\}$
// Get all charts within ρ of q_a
3: charts $\leftarrow$ WITHINRADIUS($q_a, \mathcal{A}_\mathcal{M}, \rho$)
4: **for** $C_q \in$ charts **do**
5: $u_q \leftarrow \psi_{C_q}^{-1}(q_a)$
6: $d \leftarrow \left\| \phi_{C_q}(u_q) - q_a \right\|_2$
// Check if C_q is a closer, valid chart
7: **if** $d <$ best **and** $u_q \in P_q$ **then**
8: best $\leftarrow d$, $C \leftarrow C_q$
9: **return** C

Input q_a, the configuration to get a chart for, and $\mathcal{A}_\mathcal{M}$, the current atlas of charts

Output C, the best (distance-wise) chart for q_a. If there are no valid charts for q_a, a new chart is created using q_a as the origin.

1: **procedure** GETCHART(q_a, $\mathcal{A}_\mathcal{M}$)
// Check if chart already exists
2: $C_a \leftarrow$ FINDCHART($q_a, \mathcal{A}_\mathcal{M}$)
3: **if** $C_a \neq \{\}$ **then**
4: **return** C_a
// Create new chart (see Eq. 2)
5: $C_a \leftarrow (q_a, \Phi(q_a))$
// Generate separating halfspaces (optional)
6: **if** separate charts **then**
7: adjacent $\leftarrow$ WITHINRADIUS($q_a, \mathcal{A}_\mathcal{M}, 2\rho$)
8: **for** $C_q \in$ adjacent **do**
9: COMPUTEHALFSPACES(C_a, C_q)
10: $\mathcal{A}_\mathcal{M} \leftarrow \mathcal{A}_\mathcal{M} \cup \{C\}$
11: **return** C



Sampler(SVM)

Alg. 6. The method SAMPLEATLAS is implemented to sample a chart from the collection of charts. Generally, this is uniform sampling of the collection of charts. As a heuristic improvement, bias can be introduced to SAMPLEATLAS in order to do weighted sampling (ensuring that each chart always has a positive probability of being sampled). This is done in TB-RRT, where charts are biased by their distance to the goal (Kim et al. 2016). SAMPLEBALL(C, q, r) samples a point within radius r of q within the chart C's space. Not shown in Alg. 6 are failure conditions due to ψ_C not converging; in this case, the center of an existing chart is returned.

$\beta \geq 1$ is an exploration parameter that allows search to extend from current applicable charts, so that there is a chance of generating new configurations outside of the current collection of chart's validity regions. Higher values of β have potential to generate more invalid samples, but will also cause the manifold to be more rapidly explored.

Algorithm 6 Continuation-based Sampling

Input $\mathcal{A}_\mathcal{M}$, the current atlas of charts.
Output $q \in \mathcal{M}$, a random configuration.

1: **procedure** SAMPLE($\mathcal{A}_\mathcal{M}$)
2: **do**
// Uniformly choose a chart
3: $C \leftarrow$ SAMPLEATLAS($\mathcal{A}_\mathcal{M}$)
4: $u \leftarrow$ SAMPLEBALL($C, \mathbf{0}, \rho\beta$)
5: **while** $u \notin P_C$ **or** $\neg\psi_C(u)$
6: **return** $\psi_C(u)$

Input q, the point to sample nearby, radius $r \in \mathbb{R}^+$, and $\mathcal{A}_\mathcal{M}$, the current atlas of charts.
Output $q \in \mathcal{M}$, a random configuration.

1: **procedure** NEIGHBORHOODSAMPLE(q, r $\mathcal{A}_\mathcal{M}$)
2: **do**
3: $C \leftarrow$ GETCHART($q, \mathcal{A}_\mathcal{M}$)
4: $u \leftarrow$ SAMPLEBALL($C, \psi_C^{-1}(q), r\beta$)
5: **while** $\neg\psi_C(u)$
6: **return** $\psi_C(u)$

可能failed

Integrator(LPM)

1: **procedure** LAZYINTEGRATE(q_a, q_b, $\mathcal{A}_\mathcal{M}$)
2: $q_1 \leftarrow q_a$, $j \leftarrow 1$
3: $C_j \leftarrow \text{GETCHART}(q_j, \mathcal{A}_\mathcal{M})$
4: $u_j \leftarrow \psi_{C_j}^{-1}(q_j)$ *// q_j projected into C_j*
5: $u_b \leftarrow \psi_{C_j}^{-1}(q_b)$ *// goal q_b projected into C_j*
6: **while** $\|u_j - u_b\|_2 > s$ **do**
// Interpolate in chart space
7: $u_{j+1} \leftarrow u_j + s(u_b - u_j)/\|u_b - u_j\|_2$
8: $q_{j+1} \leftarrow \varphi_{C_j}(u_{j+1})$ C-space
9: $j \leftarrow j + 1$
// Check if u_j is close to manifold
10: **if** $\|\varphi_{C_j}(u_j) - q_j\|_2 > \varepsilon$???
11: **or** $u_j \notin P_q$ **then**
// Create or get new valid chart
12: $q_j \leftarrow \psi_{C_j}(u_j)$ *// project to $\mathcal{M}$*
13: $C_j \leftarrow \text{GETCHART}(q_j, \mathcal{A}_\mathcal{M})$
14: $u_j \leftarrow \psi_{C_j}^{-1}(q_j)$
15: $u_b \leftarrow \psi_{C_j}^{-1}(q_b)$
16: **return** $\{q_k\}_{k=1}^{j}$

Efficient && accuracy

Algorithm 7 Continuation Integrator

Input $q_a \in \mathcal{M}$, an initial configuration, $q_b \in \mathcal{Q}$, a goal configuration, and $\mathcal{A}_\mathcal{M}$, the current atlas of charts.
Output $\{q_k\}_{k=1}^{j}$ such that $\forall k, q \in \mathcal{M}$. Upon success, $\|q_b - q_j\|_2 \leq s$.

1: **procedure** INTEGRATE(q_a, q_b, $\mathcal{A}_\mathcal{M}$)
2: $q_1 \leftarrow q_a$, $j \leftarrow 1$
3: $C_j \leftarrow \text{GETCHART}(q_j, \mathcal{A}_\mathcal{M})$
4: $u_j \leftarrow \psi_{C_j}^{-1}(q_j)$ *// q_j projected into C_j*
5: $u_b \leftarrow \psi_{C_j}^{-1}(q_b)$ *// goal q_b projected into C_j*
6: **while** $\|u_j - u_b\|_2 > s$ **do**
// Interpolate in chart space
7: $u_{j+1} \leftarrow u_j + s(u_b - u_j)/\|u_b - u_j\|_2$
8: $q_{j+1} \leftarrow \psi_{C_j}(u_{j+1})$ manifold
9: $d \leftarrow \|q_{j+1} - q_j\|_2$
// Check distortion & progress
10: **if** $d > \lambda s$ or $d < s$ **then**
11: **break**
12: $j \leftarrow j + 1$
// Check if u_j is in validity region
13: **if** $\|\varphi_{C_j}(u_j) - q_j\|_2 > \varepsilon$
14: **or** $s/d < \cos\alpha$
15: **or** $\|u_j\|_2 > \rho$
16: **or** $u_j \notin P_q$ **then**
// Create or get new valid chart
17: $C_j \leftarrow \text{GETCHART}(q_j, \mathcal{A}_\mathcal{M})$
18: $u_j \leftarrow \psi_{C_j}^{-1}(q_j)$
19: $u_b \leftarrow \psi_{C_j}^{-1}(q_b)$
20: **return** $\{q_k\}_{k=1}^{j}$

- Comments: These methods do not generate a “true” geodesic(the shortest curve between two points); these methods simply attempt to efficiently connect two points on the manifold, approximating the geodesic. As the manifold is locally Euclidean, as the distance between configurations lessens the curve generated by these methods approaches the true geodesic

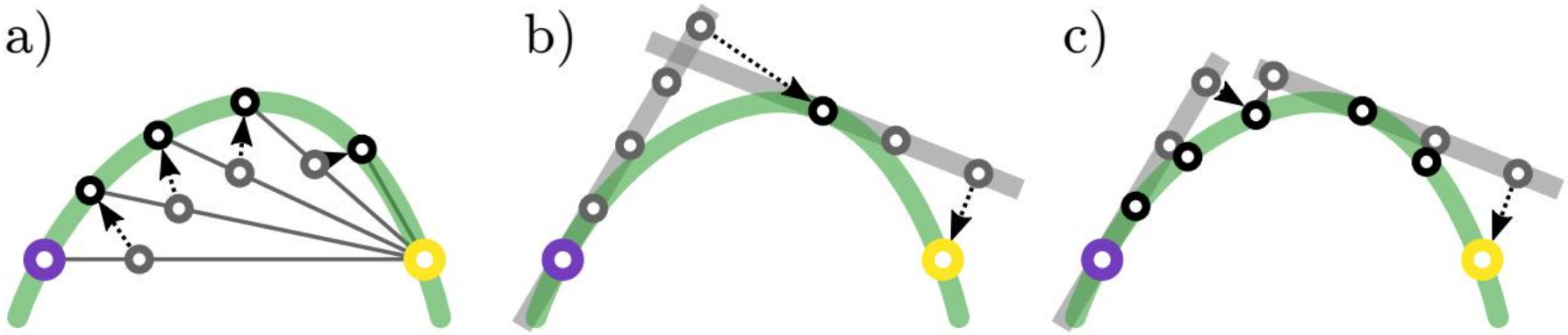


Figure 3. Projection-, tangent bundle-, and atlas-based geodesic interpolation. Between a start (purple) and goal (yellow) points on the implicit manifold (green), the discretized geodesic is computed (black). **a)** Projection-based (CBIRRT2). Small extensions are taken (grey) and projected using a projection operator (arrow). **b)** Tangent bundle-based (TB-RRT). The manifold is lazily evaluated with tangent spaces (grey), projecting when necessary. **c)** Atlas-based (AtlasRRT). Tangent spaces are traversed, projecting at every step.

Sample-based planner Assumption

- smooth manifold
- No singularity
- Single-modal manifold, do not handle multi-modal

即使不满足这两个约束，也可以用，假装不存在！！！！

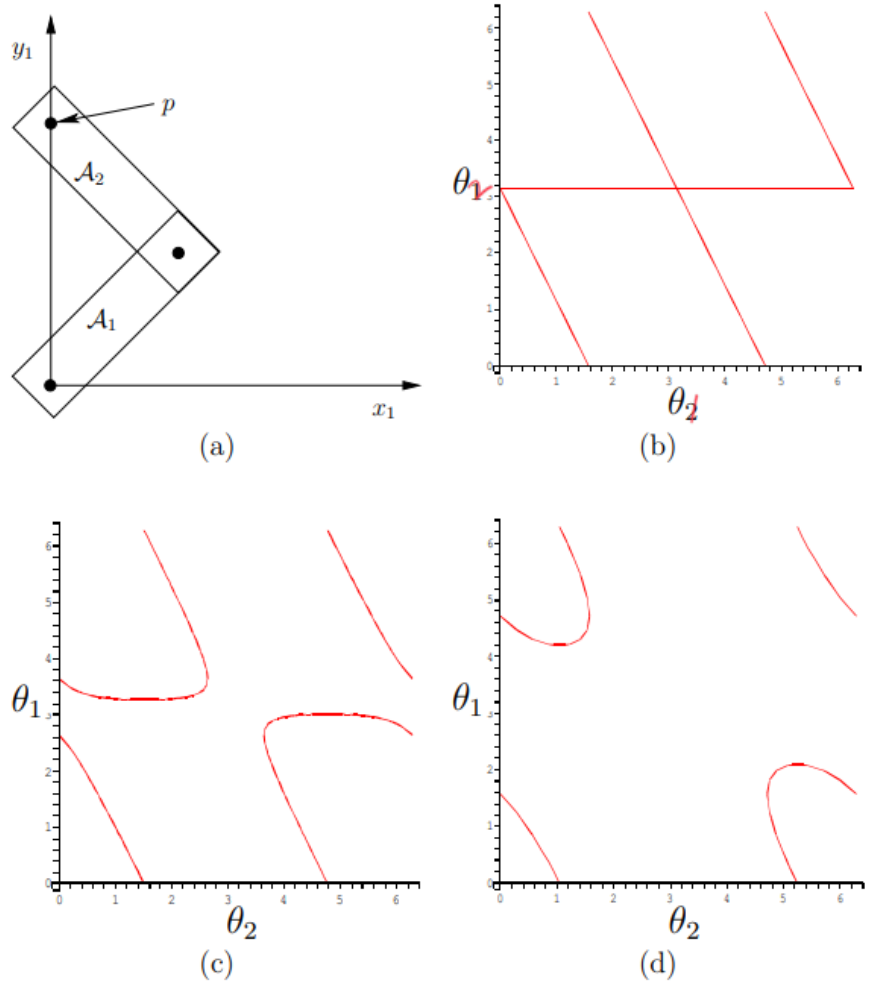


Figure 4.23: A single constraint was added to the point p on $\mathcal{A}_2$, as shown in (a). The curves in (b), (c), and (d) depict the variety for the cases of $f_1 = 0$, $f_1 = 1/8$, and $f_1 = 1$, respectively.

Constraint examples:

Query pair (qi, qg), 生成Z轴向上的路径

Projection:

VSM: 在SE(3)上sample vertex v, v 满足z轴约束，output: q = IK(v)

LPM: 找到一个距离q最近的点q-near，在q-near到q方向生成固定step点q-new，并满足约束。 q-near q-new在SE(3) integrate {vk}，每个vk，对应IK选解，选取q变化最小的解，生成{qk}

Draw Back：IK计算慢？IK计算失败？